

Neural Language Models

Complete Study Notes

Language Modeling · RNNs · LSTMs · GRUs · Seq2Seq · Attention

Based on: LLMs: Introduction and Recent Advances — Prof. Tanmoy Chakraborty, IIT Delhi

TABLE OF CONTENTS

1. Language Modeling — What & Why
2. Fixed-Window Neural Language Model
3. Recurrent Neural Networks (RNN)
4. Training RNN Language Models
5. Vanishing & Exploding Gradients
6. Long Short-Term Memory (LSTM)
7. Gated Recurrent Units (GRU) & LSTM vs GRU
8. Sequence-to-Sequence (Seq2Seq) Modeling
9. Decoding Strategies: Greedy, Beam Search
10. Attention Mechanism
11. Self-Attention & Transformers (Intro)
12. Quick-Reference Summary

1. Language Modeling — What & Why

Definition: A Language Model (LM) is a system that assigns a probability to a sequence of words, or equivalently, predicts what word comes next given a preceding context.

Why is it needed?

- Foundation for virtually every NLP task: translation, summarisation, question-answering, dialogue.
- Lets us rank candidate sentences ('the cat sat on the mat' is more probable than 'mat the on sat cat the').
- Enables text generation by repeatedly sampling the next word.

The probability chain rule:

$$P(w_1, \dots, w_T) = P(w_1) \times P(w_2 | w_1) \times \dots \times P(w_T | w_{T-1}, \dots, w_1) = \prod P(w_t | w_{t-1}, \dots, w_1)$$

■ Concrete Example

Sentence: 'I like pizza with loads of ____'

LM assigns probabilities:

- $P(\text{cheese} | \text{I like pizza with loads of}) \approx 0.55$ ✓ high
- $P(\text{corn} | \text{I like pizza with loads of}) \approx 0.10$
- $P(\text{tree} | \text{I like pizza with loads of}) \approx 0.001$ ✗ low

The model picks 'cheese' because it appears most often after such context in training data.

Evaluation — Perplexity (PP):

Lower perplexity = better model. PP is the exponentiated average negative log-likelihood:

$$PP = P(w_1, \dots, w_N)^{-1/N} = \exp(-1/N \sum \log P(w_t | \text{context}))$$

Intuition: PP ≈ 'how many equally-likely words could follow on average'. A trigram model on news text might have PP ≈ 74; a good neural LM ≈ 30.

N-gram models vs Neural models:

Aspect	N-gram LM	Neural LM
Sparsity	Severe – unseen n-grams get 0 prob	None – embeddings generalise
Storage	Huge count tables	Compact weight matrices
Context	Fixed short window (n-1 words)	Can be long (RNN/Transformers)
Generalisation	Weak	Strong (word embeddings)

2. Fixed-Window Neural Language Model

Problem it solves: N-gram models have sparsity (unseen n-grams) and storage problems. A small neural network with word embeddings can smooth over these issues.

Architecture (Bengio et al., 2003):

- Take the previous n words as one-hot vectors $x^{(1)}, \dots, x^{(n)}$.
- Look up their embeddings via embedding matrix $E \rightarrow$ concatenate into vector $e = [e^{(1)}; \dots; e^{(n)}]$.
- Pass through hidden layer: $h = f(We + b_1)$.
- Compute output distribution: $\hat{y} = \text{softmax}(Uh + b_2) \in \mathbb{V}$.

Step-by-step worked example:

■ Example: predict word after 'the students opened their'

Input window ($n=4$): the | students | opened | their

Step 1 — Embed each word (say $\text{dim}=4$):

$x^{(1)} = \text{"the"} \rightarrow e^1 = [0.2, -0.1, 0.5, 0.3]$... (4 embeddings concatenated $\rightarrow$ 16-dim vector e)

Step 2 — Hidden layer (say $\text{hidden_size}=20$):

$h = \tanh(W \cdot e + b_1)$ where W is 20×16

Step 3 — Output (vocab size=50,000):

$\hat{y} = \text{softmax}(U \cdot h + b_2) \rightarrow P(\text{books})=0.35, P(\text{exams})=0.28, \dots$

Step 4 — Predict: $\text{argmax}(\hat{y}) = \text{'books'}$

Improvements over n-gram LMs:

- No sparsity problem — unseen combinations still get probability.
- No need to store all observed n-grams — just the weight matrices.

■■ Remaining Problems

- Fixed window is too small — cannot capture long-range dependencies.
- Enlarging the window enlarges W — parameters grow linearly.
- $x^{(1)}$ and $x^{(2)}$ are multiplied by completely different weight columns $\rightarrow$ no symmetry; the model cannot share what it learned about position 1 with position 2.
- We need an architecture that handles **any-length input** with shared weights!

3. Recurrent Neural Networks (RNN)

Core idea: Apply the *same* weight matrix W at every time step, passing a hidden state h forward as a form of memory.

Update equation:

$$h^{(t)} = \sigma(W_h \cdot h^{(t-1)} + W_e \cdot e^{(t)} + b_1)$$

$$\mathbf{\hat{y}}^{(t)} = \text{softmax}(U \cdot h^{(t)} + b_2)$$

Where:

- $h^{(t)}$ — hidden state at time t (memory of what has been seen so far)
- $e^{(t)} = E \cdot x^{(t)}$ — embedding of current input word
- W_h, W_e, U — learned weight matrices (shared across all time steps)
- σ — activation (typically tanh or ReLU)

■ Step-by-Step RNN Forward Pass

Sentence: 'I am going' → predict next word

Vocabulary: {I:0, am:1, going:2, ...} | hidden_size = 3 | embed_dim = 2

t=1 $x=(1,0,0,...) \rightarrow e_1=[0.6, 0.1]$

$h_1 = \tanh(W_h \cdot h_0 + W_e \cdot e_1 + b)$

h_0 is initialised to 0; result e.g. $h_1=[0.3, 0.7, -0.1]$

t=2 $x=(0,1,0,...) \rightarrow e_2=[-0.2, 0.9]$

$h_2 = \tanh(W_h \cdot h_1 + W_e \cdot e_2 + b)$

t=3 similarly h_3 is computed from h_2 and e_3 .

Final output: $\mathbf{\hat{y}} = \text{softmax}(U \cdot h_3) \rightarrow$ distribution over vocab.

Advantage	Disadvantage
■ Can process any-length input	■ Recurrent computation is inherently sequential (slow)
■ Model size constant regardless of sequence length	■ Hard to capture information from many steps back
■ Same weights → translation-invariant processing	■ Vanishing / Exploding gradients during backprop
■ Theoretically captures long-range dependencies	■ In practice, short-term memory dominates

4. Training RNN Language Models

Loss function: Cross-entropy between predicted distribution $\hat{\mathbf{y}}^{(t)}$ and the true next word $y^{(t)}$ (one-hot for $x^{(t+1)}$):

$$J^{(t)}(\theta) = \text{CE}(\mathbf{y}^{(t)}, \hat{\mathbf{y}}^{(t)}) = -\log \hat{\mathbf{y}}_{x^{(t+1)}}^{(t)}$$

$$J(\theta) = (1/T) \sum_{t=1}^T J^{(t)}(\theta)$$

Teacher Forcing: During training, we always feed the **true** previous word as input to the next time step (not the model's own prediction). This stabilises training.

Backpropagation Through Time (BPTT): The gradient of loss $J^{(4)}$ w.r.t. W_h sums contributions across all timesteps:

$$\partial J^{(4)} / \partial W_h = \partial J^{(4)} / \partial W_h|_{t=4} + \partial J^{(4)} / \partial W_h|_{t=3} + \partial J^{(4)} / \partial W_h|_{t=2} + \dots$$

■ Why mini-batches?

Computing loss across the entire corpus at once would require $O(T)$ memory for a corpus of T tokens — infeasible. Instead we use Stochastic Gradient Descent (SGD): pick a batch of sentences, compute loss + gradients, update weights, repeat.

5. Vanishing & Exploding Gradients

When gradients are backpropagated through T time steps they are multiplied by W_h repeatedly. If the largest singular value of W_h is:

- < 1 → gradients shrink exponentially → **vanishing gradient**
- > 1 → gradients grow exponentially → **exploding gradient**

■ Why Does it Matter?

Task: complete 'When she tried to print her tickets, she found that the printer was out of toner ...
After installing the toner she finally printed her ____'

The word 'tickets' appears 7+ steps earlier.

If gradient vanishes by the time we reach step 7, the model never learns that 'tickets' is the answer
→ it might predict 'toner' instead.

RNN-LM is therefore poor at long-range dependencies in practice.

Solutions:

Problem	Fix
Exploding gradient	Gradient clipping: if $\ \nabla\ > \text{threshold}$, rescale it.
Vanishing gradient	LSTM / GRU: separate memory cell with additive updates.
Vanishing (deep nets)	Residual / skip connections (linear pass-through).
General vanishing	Attention mechanisms — direct shortcut to past states.

6. Long Short-Term Memory (LSTM)

Problem it solves: Vanilla RNN constantly overwrites its hidden state → hard to retain information over many steps. LSTM adds a *cell state* $c^{(t)}$ that information can be added to or removed from through learnable gates.

Three gates + cell update (all vectors of length n):

Forget gate $f^{(t)}$:

$$\sigma(w_f h^{t-1} + u_f x^t + b_f)$$

→ How much of the PREVIOUS cell state to keep. 0 = forget all, 1 = keep all.

Input gate $i^{(t)}$:

$$\sigma(w_i h^{t-1} + u_i x^t + b_i)$$

→ How much of the NEW candidate cell content to write.

Output gate $o^{(t)}$:

$$\sigma(w_o h^{t-1} + u_o x^t + b_o)$$

→ How much of the cell state to expose as hidden state.

New cell content $c^{(t)}$:

$$\tanh(w_c h^{t-1} + u_c x^t + b_c)$$

→ Candidate content to potentially write to cell (uses tanh, not sigmoid).

Cell and hidden state updates:

$$c^{(t)} = f^{(t)} \blacksquare c^{(t-1)} + i^{(t)} \blacksquare c^{(t)}$$

$$h^{(t)} = o^{(t)} \blacksquare \tanh(c^{(t)})$$

■ Intuitive LSTM Example

Task: language model for the sentence 'The chef who I met in Paris cooked a great meal.'

When processing 'chef': input gate fires strongly → writes 'subject=chef' to cell.

During 'who I met in Paris' (relative clause): forget gate stays at ~1 (keep chef), input gate low (ignore relative clause for subject agreement).

At 'cooked': output gate reads cell to recover 'chef' for subject-verb agreement → correctly predicts singular verb.

The cell state acts as long-term memory that survives many timesteps.

Why does LSTM fix vanishing gradients? The cell state update $c^{(t)} = f \blacksquare c^{(t-1)} + i \blacksquare c^{(t)}$ is *additive* (not multiplicative like vanilla RNN). Gradients can flow back through the + operation without repeated multiplication by W, providing an unobstructed highway for gradients.

7. Gated Recurrent Units (GRU) & LSTM vs GRU

GRU (Cho et al., 2014) simplifies LSTM by merging cell and hidden state and using only two gates:

Update gate $u^{(t)}$:

$$u^{(t)} = \sigma(w_u h^{t-1} + U_u x^t + b_u)$$

Controls what fraction of the previous hidden state to keep vs update.

Reset gate $r^{(t)}$:

$$r^{(t)} = \sigma(w_r h^{t-1} + U_r x^t + b_r)$$

Controls how much of past hidden state to use for new candidate.

New candidate hidden state:

$$h_{\square}^{(t)} = \tanh(w_h (r^{(t)} \square h^{(t-1)}) + U_h x^{(t)} + b_h)$$

Hidden state update:

$$h^{(t)} = (1 - u^{(t)}) \square h^{(t-1)} + u^{(t)} \square h_{\square}^{(t)}$$

Note: the update gate simultaneously controls preservation and updating — a single gate replacing LSTM's forget + input gates.

LSTM vs GRU Comparison:

Property	LSTM	GRU
Gates	3 (forget, input, output)	2 (update, reset)
Separate cell state	Yes	No
Parameters	More	Fewer
Speed	Slower	Faster
Long dependencies	Usually better	Comparable
Recommended when	Long deps / lots of data	Faster training needed

■ Rule of Thumb

Start with LSTM as the default. Switch to GRU when training speed or parameter efficiency is the priority. No single model dominates across all datasets.

8. Sequence-to-Sequence (Seq2Seq) Modeling

What it is: A neural architecture that maps an input sequence $x = (x_1, \dots, x_m)$ to an output sequence $y = (y_1, \dots, y_n)$ of *different* length. Key application: Neural Machine Translation (NMT).

Two-component design:

- **Encoder RNN:** reads source sentence word by word, producing a sequence of hidden states. The final hidden state h_{enc} is the 'encoding' of the sentence.
- **Decoder RNN:** a conditional language model. At each step it takes the previous output word and the previous decoder hidden state and generates the next word.

Conditional probability:

$$P(y|x) = \prod_{t=1}^T P(y_t \mid y_1, \dots, y_{t-1}, x)$$

■ Translation Example (Hindi → English)

Source: '■■ ■■■■■■ ■■ ■■' (ham baajaar gaye the)

Encoder reads each word → final hidden state encodes full sentence meaning.

Decoder: → 'we' → 'had' → 'gone' → 'to' → 'the' → 'market' →

At each decoder step: new hidden state = $f(\text{prev_hidden}, \text{embedding}(\text{prev_output}))$

Output word = $\text{argmax softmax}(U \cdot \text{decoder_hidden} + b)$

Training: Teacher forcing + end-to-end backpropagation. Total loss = $J_1 + J_2 + \dots + J_T$. Gradients flow through both decoder and encoder.

■■ The Bottleneck Problem

The entire source sentence must be compressed into a single fixed-size vector (the encoder's final hidden state). For long sentences, this is a severe information bottleneck — early source words are 'forgotten'.

Solution: Attention Mechanism (Section 10).

9. Decoding Strategies

Greedy Decoding:

At each decoder step, pick the single most probable word: $y_t = \operatorname{argmax} P(y|y_{1..t-1}, x)$. Simple but suboptimal because early mistakes cannot be undone.

■ Greedy Decoding Problem

Source: 'il a m'entarté' → correct translation: 'he hit me with a pie'

Greedy output: 'he' → 'hit' → 'a' → (wrong!)

Once 'a' is chosen instead of 'me', all subsequent words are conditioned on the mistake.

Beam Search Decoding:

At each step, keep the top k (beam size) most probable partial sequences (hypotheses). Each hypothesis is scored by its log probability:

$$\text{score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P(y_i | y_1, \dots, y_{i-1}, x)$$

■ Beam Search Walk-Through (k=2)

Step 0:

Step 1: keep top 2 words:

he [-0.7] I [-0.9]

Step 2: expand each → 4 hypotheses, keep top 2:

he hit [-1.7] I was [-1.6] (discard: he struck, I got)

Step 3: expand again → keep top 2:

he hit me [-2.5] I was hit [-2.9]

... continue until or max length.

Final answer after length-normalisation: 'he hit me with a pie' ✓

Stopping & final selection:

- A hypothesis is complete when it produces .
- Continue until T steps or n completed hypotheses.
- Length-normalise final score: $(1/t) \sum \log P(y_i|...)$ — prevents preference for short outputs.

Strategy	Quality	Cost	Notes
Greedy	Low	$O(1)$ per step	Can't undo mistakes
Beam (k)	Medium	$O(k)$ per step	Not globally optimal
Exhaustive	Optimal	$O(V^T)$	Completely infeasible

10. Attention Mechanism

Problem solved: The single encoder vector bottleneck loses information for long sentences. Attention gives the decoder a *direct* connection to every encoder state at every decoder step.

General definition: Given a set of vector *values* (encoder hidden states) and a vector *query* (decoder hidden state), attention computes a *weighted sum* of the values, where weights reflect how relevant each value is to the query.

Step-by-Step Attention Computation:

Step 1 — Compute attention scores:

$$\mathbf{e}^t = [\mathbf{s}_t^T \mathbf{h}_1, \dots, \mathbf{s}_t^T \mathbf{h}_N] \in \mathbb{R}^N$$

→ Dot product of current decoder state s_t with each encoder hidden state h_i . High score = decoder and encoder 'agree' on what information is relevant.

Step 2 — Normalise (softmax):

$$\alpha^t = \text{softmax}(e^t) \in \mathbb{R}^N \text{ (sums to 1)}$$

→ Convert raw scores to a probability distribution (the 'attention distribution').

Step 3 — Compute context vector:

$$\mathbf{a}_t = \sum_{i=1}^N \alpha_i^t \cdot \mathbf{h}_i \in \mathbb{R}^h$$

→ *Weighted average of encoder hidden states* — 'what the decoder chooses to focus on'.

Step 4 — Concatenate and predict:

$$[a_t ; s_t] \in \mathbb{R}^{2h}$$

→ Concatenate context vector with decoder state, then use for output prediction as before.

■ Attention in Action

Translating '■■ ■■■■■■ ■■ ■■' → English, generating word 'market' (t=6):

Attention distribution: $\alpha_1(\blacksquare\blacksquare)=0.05$, $\alpha_2(\blacksquare\blacksquare\blacksquare\blacksquare\blacksquare\blacksquare)=0.82$, $\alpha_3(\blacksquare\blacksquare)=0.08$, $\alpha_4(\blacksquare\blacksquare)=0.05$

The model strongly focuses on **market/bazaar** ('market/bazaar') when producing 'market'.

Context vector $\mathbf{a}_6 \approx 0.82 \cdot \mathbf{h}_2 + \text{small contributions from other states.}$

This alignment is learned automatically — no hand-labelled word alignments needed!

Attention variants:

Name	Formula	Reference
Original (additive)	$a(q,k) = w^T \tanh(W[q;k])$	Bahdanau 2015
Bilinear product	$a(q,k) = q^T W k$	Luong 2015
Dot product	$a(q,k) = q^T k$	Luong 2015

Scaled dot product	$a(q,k) = q^T k / \sqrt{ k }$	Vaswani 2017
--------------------	-------------------------------	--------------

■ Why Attention is Great

- Solves the bottleneck — decoder can directly look at any encoder state.
- Helps vanishing gradient — short-circuits long paths with direct connections.
- Provides interpretability — the attention matrix shows word alignment for free.
- Significantly improves NMT BLEU scores vs plain seq2seq.

11. Self-Attention & Transformers (Introduction)

Self-Attention: Instead of the decoder attending to the encoder, each position in a *single* sequence attends to *all* other positions in that same sequence. Query, Key, and Value all come from the same set of vectors.

Problem with RNNs that self-attention solves:

- **Non-parallelism:** RNNs must process tokens sequentially — cannot parallelise across positions.
- **Linear interaction distance:** Information from position 1 must pass through T-1 steps to reach position T.
- **Bottleneck:** Seq2seq bottleneck (even with attention, RNN encoder is sequential).

Self-Attention Equations (for position i):

$$q_i = Q^T x_i \quad k_i = K^T x_i \quad v_i = V^T x_i$$

$$a_i = \sum_j \text{softmax}(q_i \cdot k_j / \sqrt{d_k}) \cdot v_j$$

All queries are computed in parallel (matrix ops) → O(1) sequential steps vs O(n) for RNNs.

Transformer Encoder Block (one layer):

- **Multi-Head Self-Attention (MSA):** h independent attention heads, each with its own Q,K,V matrices. Concatenate outputs → final attention output captures different types of relationships.
- **Position-wise Feed-Forward Network (FFN):** Applied independently to each position: $\text{FFN}(x) = M_2^T f(M_1^T x + b_1) + b_2$. Adds non-linearity and position-specific transformations.
- **Residual connections + Layer Norm:** Each sub-layer: $\text{output} = \text{LayerNorm}(x + \text{SubLayer}(x))$.
- **Positional Encodings:** Since self-attention is permutation-invariant, sinusoidal (or learned) positional embeddings p_i are added to x_i .

■ Self-Attention Example

Sentence: 'I went to the market' → 5 tokens

For token 'market' (position 5), self-attention computes:

$$e_5 = [q_5 \cdot k_1, q_5 \cdot k_2, q_5 \cdot k_3, q_5 \cdot k_4, q_5 \cdot k_5]$$

After softmax: $\alpha = [0.05, 0.10, 0.10, 0.10, 0.65]$ (mostly self + 'went')

$$a_5 = 0.05v_1 + 0.10v_2 + \dots + 0.65v_5$$

All 5 positions computed in parallel in a single matrix multiplication!

12. Quick-Reference Summary

Model / Concept	Problem Solved	Key Idea	Limitation
N-gram LM	Statistical LM	Count-based conditional probs	Sparsity; huge storage
Fixed-Window NN	Sparsity + storage	Embeddings + MLP	No symmetry; fixed context
RNN-LM	Variable-length context	Shared weights, recurrent hidden state	Sequential; vanishing grad
LSTM	Vanishing gradient	Cell state + 3 gates (additive update)	Expensive; still sequential
GRU	Simpler LSTM	2 gates, merged cell/hidden state	Similar to LSTM
Seq2Seq	Variable-length output	Encoder RNN + Decoder RNN	Information bottleneck
Attention	Seq2Seq bottleneck	Weighted sum of encoder states	Added complexity; seq enc.
Self-Attention	Parallelism + long deps	Every token attends to all others	$O(n^2)$ memory/compute
Transformer	All RNN limitations	Multi-head SA + FFN + residuals	Needs positional encodings

Key Formulas at a Glance:

Perplexity	$PP = \exp(-1/N \cdot \sum \log P(w_t \text{context}))$
RNN hidden state	$h^t = \sigma(w_h h^{t-1} + w_e e^t + b)$
LSTM cell update	$c^t = f^t \blacksquare c^{t-1} + i^t \blacksquare c^t$
Attention scores	$e^t = [s_t^T h_1, \dots, s_t^T h_N]$
Context vector	$a_t = \sum \text{softmax}(e^t)_i \cdot h_i$
Scaled dot-product	$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$
Beam Search score	$\text{score}(y_1 \dots y_t) = \sum \log P(y_i \text{prev}, x)$

Notes compiled from: 'LLMs: Introduction and Recent Advances' — Prof. Tanmoy Chakraborty, IIT Delhi | Wiley: Introduction to Large Language Models